ISW-G9-4K1-2026

Trabajo Práctico N.º 6 (TP6)

Decisiones de Diseño y Organización del Desarrollo

Documento del **Grupo 9 (Curso 4K1 - Año 2026)** de la materia **Ingeniería y Calidad de Software (UTN-FRC)**.

Proyecto: EcoHarmony Park
Historia de usuario: Comprar entradas

Integrantes del Grupo

- **Nombre:** Calderón, Ana Victoria - **Legajo:** 402851 - **Usuario:** viccalderon
- **Nombre:** Díaz, Brisa Abigaíl - **Legajo:** 97287 - **Usuario:** BrisaDiaz
- **Nombre:** Escudero Olivera, Matías Rubén - **Legajo:** 402434 - **Usuario:** escuderomann
- **Nombre:** Fuertes Galera, Emiliano - **Legajo:** 402186 - **Usuario:** emolianito
- **Nombre:** Galetto, Fiorella - **Legajo:** 400305 - **Usuario:** fiorellag26
- **Nombre:** Gorosito, Tamara - **Legajo:** 90740 - **Usuario:** tamigorosito234-ai
- **Nombre:** Guevara Lopez, Joaquin - **Legajo:** 300543 - **Usuario:** joaquinsinho
- **Nombre:** Juri Romero, Agustina - **Legajo:** 95317 - **Usuario:** agustinajuri18
- **Nombre:** Linaza, Camila - **Legajo:** 402370 - **Usuario:** camilinaza08
- **Nombre:** Maccio, Tomás Agustín - **Legajo:** 400057 - **Usuario:** maccio-bit
- **Nombre:** Pizarro, José Francisco - **Legajo:** 402123 - **Usuario:** franpi19
- **Nombre:** Rigonatto, Enzo - **Legajo:** 400629 - **Usuario:** rigonattoenzo
- **Nombre:** Torrens Valsagna, Francisco - **Legajo:** 401939 - **Usuario:** frantorrens

Objetivo

El objetivo de este documento es **establecer y justificar las decisiones de diseño y organización del trabajo** adoptadas por el Grupo 9 para desarrollar la historia de usuario **«Comprar entradas»** del proyecto EcoHarmony Park, correspondiente al TP6 de Ingeniería y Calidad de Software.

Para ello, se describen la metodología de desarrollo, la organización y seguimiento de tareas, las convenciones de trabajo en el repositorio y las herramientas utilizadas para implementar el frontend, el backend y las pruebas.

Además, se establece cómo el diseño de tests guía el desarrollo mediante TDD en el backend, manteniendo un trabajo organizado, progresivo y coordinado entre los integrantes. El alcance funcional comprende la selección de entradas por un visitante registrado con sesión activa, el cálculo del monto, el registro persistente de la compra, la elección del medio de pago y la confirmación correspondiente.

El registro de usuarios, la autenticación real, el cobro en boletería y la inscripción a actividades pertenecen a otras historias de usuario.

1. Metodología de trabajo

Para desarrollar la historia de usuario **«Comprar entradas»** de EcoHarmony Park, decidimos adoptar un enfoque de trabajo **Just-in-Time (JIT)**, acompañado de **Test-Driven Development (TDD)** para el backend.

La idea es avanzar progresivamente, implementando las funcionalidades a medida que sean necesarias y evitando desarrollar anticipadamente comportamientos que todavía no fueron definidos o validados.

Desarrollo Just-in-Time

Desarrollamos únicamente las funcionalidades necesarias para cumplir con los requerimientos abordados en cada momento.

Este criterio también se aplica a la **instalación de librerías y dependencias**: incorporamos cada herramienta cuando es necesaria para implementar una funcionalidad concreta, evitando instalar anticipadamente paquetes que todavía no utilizamos. Identificar una tecnología como alternativa inicial no implica incorporarla al proyecto.

Las reglas de negocio se implementan a partir de las definiciones de los Product Owners (PO). Las decisiones técnicas se resuelven dentro del alcance del TP6: aplicación web local, sesión y usuarios de prueba, persistencia en archivos y servicios externos aislables mediante mocks.

Desarrollo guiado por pruebas (TDD)

Para el **backend** utilizamos TDD, siguiendo el ciclo **RED → GREEN → REFACTORING**:

- **RED:** escribir un test que represente el comportamiento esperado y comprobar que falle porque todavía no está implementado.
- **GREEN:** desarrollar la solución mínima necesaria para que el test pase.
- **REFACTORING:** revisar y mejorar el código, manteniendo el comportamiento validado por los tests.

Utilizamos el documento de diseño de tests como referencia para planificar e implementar progresivamente los comportamientos del backend. Cada nuevo test se incorpora respetando las dependencias funcionales y conservando la implementación acumulada, de manera que las funcionalidades previamente desarrolladas continúen funcionando correctamente.

Cuando un test **pasa desde su primera ejecución con el código existente**, lo registramos como **Verificación y regresión**. Documentamos el ciclo y explicamos por qué no se necesitó desarrollar código adicional. No inventamos un fallo RED, una implementación GREEN ni una refactorización para completar fases. El commit REFACTORING se realiza únicamente cuando existe una mejora efectiva.

Desarrollo y validación del frontend

Para el **frontend no utilizamos TDD**. Su desarrollo es incremental, siguiendo las funcionalidades y los flujos definidos para la historia de usuario y el diseño de interfaz aprobado por los PO.

La validación de la interfaz se realiza principalmente mediante **pruebas manuales**, comprobando navegación, visualización de información, validación de campos, mensajes de error, adaptación a diferentes pantallas e interacción con el backend. No se establece un proceso de tests automatizados del frontend como requisito de desarrollo.

El recorrido de compra presenta primero los datos de la visita y su resumen; **la forma de pago se selecciona después del resumen**. El resultado informa la cantidad de entradas, la fecha y el estado de la operación, diferenciando el registro de una compra del pago efectivamente aprobado.

Aplicación de la metodología

1. Seleccionar el siguiente comportamiento o funcionalidad a desarrollar.
2. Revisar los requerimientos, sus valores límite y los tests correspondientes.
3. Incorporar las dependencias necesarias siguiendo Just-in-Time.
4. Desarrollar el backend mediante TDD y el frontend de forma incremental, con validaciones manuales.
5. Ejecutar el nuevo test y las pruebas anteriores relacionadas; distinguir un comportamiento nuevo de una verificación del código existente.
6. Registrar únicamente los cambios y fases efectivamente realizados, conservando la numeración y descripción del test.
7. Integrar las funcionalidades desarrolladas y verificar manualmente el recorrido completo de compra desde la interfaz, incluyendo la comunicación con el backend y, cuando corresponda, los servicios externos de pago y correo electrónico.

2. Diseño y ejecución de tests

Para organizar el desarrollo de la historia de usuario **«Comprar entradas»**, elaboramos un documento de diseño de tests que utilizamos como **guía principal para implementar las funcionalidades**.

Documento de diseño de tests: Consultar documento

Este documento establece los comportamientos que debemos verificar, las funciones involucradas y los tests que orientan el desarrollo del backend mediante TDD. También incluye la división de tareas del frontend.

Organización del documento

La numeración de referencia llega hasta el **Test 198**:

1. **Manejo de usuarios (Tests 1–9):** validación del usuario registrado y consulta de datos de apoyo para la compra.
2. **Manejo de sesiones (Tests 10–18):** condiciones necesarias para considerar activa la sesión simulada.
3. **Manejo de fechas (Tests 19–39):** fecha actual o futura, anticipación máxima de 30 días y apertura del parque.
4. **Cantidad de entradas (Tests 40–54):** formato y límite de la cantidad solicitada.
5. **Datos de las entradas (Tests 55–77):** una edad y un tipo de pase válidos por visitante.
6. **Cálculo del monto (Tests 78–102):** precios base, descuentos, compras mixtas y valores límite.
7. **Selección del pago (Tests 103–110):** medios admitidos y estados iniciales de la compra y del pago.
8. **Persistencia y límite acumulado (Tests 111–135):** almacenamiento, recuperación, actualización y cantidad acumulada por usuario y fecha.
9. **Envío de correo (Tests 136–156):** contenido, destinatario y envío del resumen de compra.
10. **Mercado Pago (Tests 157–198):** preferencias, clientes del SDK, retorno de Checkout Pro y consulta y validación de pagos.

Además, el documento incluye una sección destinada a la **división de tareas del frontend**, donde se identifican las pantallas y funcionalidades necesarias para desarrollar la interfaz y distribuir las responsabilidades entre los integrantes del grupo.

Estructura de definición de los tests

Dentro de cada sección, los tests están numerados y organizados por comportamiento. Para cada uno se describen su objetivo, precondiciones, datos de entrada, ejecución y resultado esperado, junto con las consideraciones necesarias para orientar su implementación mediante TDD.

Durante el desarrollo se registra la aplicación efectiva del ciclo RED → GREEN → REFACTORING. Cuando un test pasa con la implementación existente, se identifica como Verificación y regresión, sin introducir modificaciones artificiales.

Esta organización permite identificar los comportamientos esperados y proporciona una referencia común para implementar las pruebas y las funcionalidades asociadas. Se trata de un documento de diseño de tests, no de una especificación formal de casos de prueba.

Pseudocódigos individuales

Los pseudocódigos individuales se elaboraron hasta la **sección 7 inclusive**, correspondiente a la selección de la forma de pago.

Los archivos conservan la nomenclatura `TP6_Pseudocodigo_Test_[Número].pdf` y `TP6_Pseudocodigo_Test_[Número].zip`, incluyendo cada ZIP su PDF correspondiente.

Para las secciones posteriores, el desarrollo se orientó directamente por las definiciones del documento de diseño de tests, sin generar pseudocódigos individuales adicionales.

La documentación distingue los commits propuestos de aquellos efectivamente realizados en el repositorio.

División de tareas del frontend

Como el frontend no se desarrolla mediante TDD, su organización se realizó de manera independiente de la numeración de tests del backend.

En el documento de diseño de tests se estableció la división de tareas correspondiente a la estructura general, navegación, calendario, selección de entradas y datos por visitante, resumen, selección del medio de pago y presentación del resultado.

Las páginas y sus scripts se distribuyeron entre los integrantes, manteniendo estilos y contratos HTTP compartidos. Su validación se realizó principalmente de forma manual y se coordinó con la integración del backend.

Esta distribución quedó registrada en el documento de diseño de tests, mientras que el seguimiento mediante Trello se concentró en las tareas de testing y desarrollo del backend.

Uso del documento durante el desarrollo

El diseño de tests es nuestra **referencia para planificar, distribuir e implementar el trabajo**. Seleccionamos los tests y tareas de cada funcionalidad y avanzamos según los comportamientos descritos y sus dependencias técnicas.

Los tests conservan su numeración y la implementación de los anteriores. Cuando un caso ya está cubierto, agregamos la prueba como verificación y regresión, sin alterar artificialmente el código para provocar un fallo.

Los cambios en reglas de negocio se contrastan con las definiciones de los PO antes de trasladarlos al código. La conexión entre servicios y pantallas se revisa durante el desarrollo del recorrido de compra.

Las verificaciones manuales del recorrido completo de compra, incluyendo la interacción con Mercado Pago Checkout Pro y la confirmación mediante correo electrónico, se realizan como actividades de integración. No se incorporan a la numeración de tests del backend ni se presentan como ciclos TDD automatizados.

3. Organización del trabajo con Trello

Acordamos utilizar **Trello** para organizar y seguir las tareas del TP6 mediante un tablero compartido.

Tablero de Trello: Acceder al tablero

El objetivo es visualizar el avance, facilitar la distribución de tareas y evitar concentrar el trabajo en los últimos días.

Estructura del tablero

El esquema de trabajo contempla tres columnas:

- **To Do:** tareas pendientes.
- **Doing:** tareas en desarrollo.
- **Done:** tareas finalizadas según los criterios establecidos.

Creamos una tarjeta por cada test correspondiente a las **primeras siete secciones del documento de diseño de tests**, utilizando su número y nombre como título. Las tarjetas se identificaron mediante etiquetas de colores para distinguir las secciones funcionales.

A partir de la sección 8, continuamos el desarrollo siguiendo el documento de diseño de tests, sin extender la creación de tarjetas individuales en Trello. De esta manera, el tablero se utilizó como herramienta de organización y seguimiento durante las primeras etapas del trabajo, mientras que el documento de diseño de tests continuó siendo la referencia para las funcionalidades posteriores.

Información de las tarjetas

Para los tests organizados mediante Trello, cada tarjeta incluye:

- Número y nombre del test.
- Etiqueta de color correspondiente a su sección.
- Fecha de vencimiento tentativa, utilizada como referencia para promover un avance progresivo y no como plazo rígido.
- Enlace al pseudocódigo correspondiente.

El vencimiento de la fecha no implica que una tarea esté terminada. Su cierre depende de la implementación, la ejecución de las pruebas correspondientes y el registro de los cambios efectivos.

Asignación y seguimiento

Trabajamos con **autoasignación según disponibilidad**. Cuando un integrante toma una tarjeta de **To Do**, se asigna como responsable y la mueve a **Doing**. Cuando cumple el criterio de finalización, la mueve a **Done**.

Antes de modificar un servicio compartido se coordina con quienes trabajan en la misma sección. Cada aporte conserva las funcionalidades acumuladas y comprueba que los nuevos cambios no alteren comportamientos existentes.

Criterios para considerar una tarea finalizada

Independientemente de que una tarea haya sido registrada en Trello o gestionada directamente a partir del documento de diseño de tests, consideramos los siguientes criterios para determinar su finalización:

- **Test unitario del backend:** la prueba está incorporada, se ejecutó junto con las anteriores relacionadas y los commits reflejan únicamente las fases efectivamente realizadas. No se exige completar tres commits si no hubo cambios GREEN o REFACTORING.
- **Verificación y regresión:** el nuevo test pasó desde su primera ejecución con la implementación existente; se documentó esta situación y se registró un único commit con ese sufijo.
- **Tarea del frontend:** la funcionalidad está implementada y se verificaron manualmente su comportamiento, presentación y comunicación con el backend cuando corresponde.
- **Conexión de componentes:** las rutas, los servicios y la interfaz intercambian los datos necesarios para completar el recorrido correspondiente, y el cambio queda registrado en Git.

Organización del frontend y fecha límite

La división de tareas del frontend se estableció en el documento de diseño de tests, donde se identificaron las pantallas y funcionalidades que debían desarrollar los integrantes del grupo. Esta distribución no se trasladó a tarjetas individuales del tablero de Trello.

Como hito interno de planificación, fijamos el **martes 20 de octubre de 2026 a las 23:59 h** para completar las tareas del frontend, reservando el **miércoles 21 de octubre de 2026** para conectar los componentes y revisar el recorrido de compra.

Estas fechas expresan la organización acordada para coordinar los aportes del equipo. La finalización de cada funcionalidad se determina por su implementación y validación, independientemente de su registro en Trello.

4. Gestión de cambios y commits

Utilizamos **Git y GitHub** para registrar los cambios y seguir la evolución del TP6.

Repositorio de GitHub: Acceder al repositorio

La convención de commits diferencia el backend del frontend, ya que TDD se aplica únicamente al primero. Los commits locales conservan sus mensajes y separación por fases aunque se envíen juntos mediante un mismo `push`.

Formato de los commits del backend

Usamos el formato:

```
Test [Número] - [Descripción del test] - [Fase]
```

- **[Número]:** número según el plan de pruebas.
- **[Descripción del test]:** descripción idéntica en los distintos commits del mismo test.
- **[Fase]:** etapa efectivamente realizada o verificación del comportamiento existente.

Las opciones para **[Fase]** son:

Fase	Significado

Fase	Significado
`RED`	Se incorpora una prueba que falla porque falta el comportamiento esperado.
`GREEN`	Se modifica la implementación para superar la prueba.
`REFACTORING`	Se mejora efectivamente la estructura sin cambiar el comportamiento.
`Verificación y regresión`	El nuevo test pasa desde su primera ejecución sin modificar la implementación.

No utilizamos prefijos como `feat:`, `test:` o `fix:` en los commits sujetos a esta convención. La descripción y la numeración se mantienen estables entre fases.

Commits de tests unitarios

Para los tests que requieren desarrollar un comportamiento nuevo registramos:

1. **RED:** incorporación del test y comprobación del fallo esperado.
2. **GREEN:** implementación mínima que permite superar la prueba, cuando existe un cambio de código.
3. **REFACTORING:** mejora del diseño, únicamente cuando se realizan modificaciones efectivas y las pruebas continúan pasando.

Ejemplo:

```
Test 13 - Validar una sesión finalizada - RED
Test 13 - Validar una sesión finalizada - GREEN
Test 13 - Validar una sesión finalizada - REFACTORING
```

El ejemplo muestra las fases posibles; **no obliga a producir tres commits por test**.

Si la prueba pasa con la implementación acumulada, corresponde un único commit:

```
Test 12 - Validar una sesión inexistente - Verificación y regresión
```

Cuando un test pasa desde su primera ejecución con la implementación existente, registramos esta situación como **Verificación y regresión**, explicando por qué no fue necesario realizar modificaciones adicionales.

En estos casos no se ejecutan artificialmente las fases RED, GREEN o REFACTORING, ni se generan commits vacíos para completar el ciclo. Se conserva la evidencia de la verificación realizada y el comportamiento previamente implementado.

Antes de cada commit se revisa el resultado de las pruebas y se agregan al área de preparación los archivos que corresponden a esa fase. Un commit propuesto en la documentación solo se considera realizado cuando existe en Git.

Commits de integración

Los cambios destinados a conectar los servicios, las rutas HTTP, el frontend y las herramientas externas se registran mediante mensajes descriptivos que identifican claramente la modificación realizada.

Estos commits no siguen la convención de numeración y fases utilizada para los tests del backend, ya que corresponden a tareas de integración, configuración o ajustes realizados durante el desarrollo.

Cuando una modificación corresponde a la implementación de un test numerado, se conserva la convención del backend, respetando su número, descripción y fase efectivamente realizada.

Commits del frontend

En el frontend no aplicamos TDD, por lo que no exigimos tres commits por tarea ni utilizamos el formato establecido para los tests del backend.

Los mensajes de commit describen de manera clara y representativa la funcionalidad implementada o la modificación realizada.

Se procura separar los cambios de interfaz de los de lógica de negocio para facilitar la revisión. Un mismo envío a GitHub puede incluir varios commits locales, conservando el historial de cada uno.

5. Estructura del repositorio

Definimos una estructura sencilla que separa frontend y backend y organiza el código del backend por responsabilidad funcional. Las rutas HTTP delegan en servicios, las validaciones y el cálculo de entradas se concentran en funciones específicas, y el acceso persistente a compras se encapsula en un Repository.

Estructura general

La organización principal del código y de las pruebas unitarias es la siguiente:

```
backend/
├── src/
│   ├── index.js                        # Inicio del servidor HTTP
│   ├── server.js                       # Aplicación Express, middlewares y rutas
│   ├ data/
│   │   └── compras.json                # Compras persistidas
│   └── services/
│       ├── ticket/
│       │   ├── ticket.service.js       # Coordinación de compra y confirmación
│       │   ├── ticket.validator.js     # Validaciones y cálculo del monto
│       │   ├── ticket.service.test.js  # Pruebas de entradas y sus validadores
│       │   ├── usuarios.mock.js
│       │   ├── tiposPase.mock.js
│       │   ├── formasPago.mock.js
│       │   └── calendario.mock.js
│       ├── sesion/
```

```
│       │   ├── sesion.service.js
│       │   └── sesion.service.test.js
│       ├── repository/
│       │   ├── compraRepository.js
│       │   └── compraRepository.test.js
│       ├── mercadopago/
│       │   ├── mercadopago.service.js   # Preferencias y verificación de pagos
│       │   ├── mercadopago.service.test.js
│       │   ├── mercadopago.client.js    # Inicialización del SDK oficial
│       │   ├── mercadopago.client.test.js
│       │   └── mercadopago.integration.js # Coordinación del inicio de Checkout
│       └── mail/
│           ├── mail.service.js
│           ├── mail.service.test.js
│           ├── confirmacionCompra.hbs
│           ├── logo.png
│           ├── preview.mail.js
│           └── preview-mail.html
├── prueba-mercadopago.js                 # Script auxiliar de uso del cliente
├── package.json
└── package-lock.json
frontend/
├── index.html
├── login.html
├── compra.html
├── pago.html
├── resultado.html
├── visitas.html
├── alimentacion.html
├── js/                                   # Interacción y solicitudes HTTP por pantalla
├── css/                                  # Estilos compartidos y por pantalla
└── assets/
    └── img/                              # Identidad visual e imágenes
```

Los tests de cada servicio se mantienen junto con su implementación. Las dependencias se registran en `package.json` y `package-lock.json` para reproducir el entorno con npm.

Organización del frontend

La carpeta **`frontend/`** reúne páginas HTML y carpetas de JavaScript, CSS e imágenes. El flujo principal se organiza en `compra.html`, `pago.html` y `resultado.html`, acompañado por la página inicial y la pantalla visual de acceso.

Cada pantalla dispone de scripts y estilos propios cuando los necesita. La navegación y la identidad visual se comparten; `calendario.js` se reutiliza para la selección de fechas y `sesion.js` proporciona la sesión de demostración para la compra.

`compra.js` solicita las reglas de calendario y los tipos de pase, genera los datos por visitante y muestra el resumen devuelto por el backend. `pago.js` envía la confirmación y redirige a la URL recibida.

`resultado.js` obtiene el identificador desde el retorno o desde la compra almacenada y consulta el estado al backend antes de presentarlo.

Las páginas de visitas y alimentación tienen alcance informativo o de demostración visual, con datos de ejemplo. La inscripción a actividades y su lógica de negocio quedan fuera de la historia **«Comprar entradas»**.

Organización del backend

Cada sección funcional tiene su carpeta, con implementación y tests ubicados juntos. Los **mocks** de usuarios, pases, medios de pago y reglas de calendario se mantienen en `ticket/`.

- **`ticket`:** coordina la compra y su confirmación, consulta los catálogos y aplica las reglas de entradas. Las validaciones y el cálculo permanecen en `ticket.validator.js`; sus tests se incorporan en `ticket.service.test.js`.
- **`sesion`:** valida las condiciones de una sesión activa.
- **`repository`:** encapsula la lectura, escritura y consulta persistente de compras.
- **`mercadopago`:** prepara preferencias, consulta pagos y verifica sus datos. La creación de clientes oficiales se separa en `mercadopago.client.js`; las funciones de negocio reciben los clientes como dependencias.
- **`mail`:** genera el contenido HTML y realiza el envío SMTP.

No se crea un archivo de implementación o de test por cada número del plan. Cada caso se agrega al archivo del servicio correspondiente, conservando la implementación acumulada.

Comunicación entre frontend y backend

`server.js` configura Express y expone la API. **`index.js`** inicia la escucha HTTP, separando el arranque del servidor de la aplicación exportada.

La comunicación utiliza **HTTP y JSON**. El frontend realiza solicitudes con `fetch`; no invoca directamente funciones internas del backend. La URL base de desarrollo es `http://localhost:3000`.

Método y ruta	Responsabilidad dentro del flujo
`GET /api/formas-pago`	Obtener las opciones Efectivo y Mercado Pago.
`GET /api/tipos-pase`	Obtener identificadores y precios base de los pases.
`GET /api/calendario/reglas`	Obtener los días semanales y las fechas fijas de cierre.
`POST /api/compra`	Recibir `sesion` y `datosCompra`, validar y devolver la compra con `montoTotal`.
`POST /api/confirmacion`	Recibir `sesion`, `compra` y `formaPago`, guardar la compra y devolver la URL de resultado o la de Checkout Pro, según el medio elegido.
`GET /api/pagos/:id/estado`	Recuperar la compra persistida y, si corresponde a Mercado Pago, consultar sus pagos y verificar una aprobación válida.

Durante el desarrollo, el frontend se ejecuta mediante un servidor estático local en el puerto 5500, mientras que el backend utiliza el puerto 3000. Para verificar el retorno desde Mercado Pago Checkout Pro, se utiliza ngrok, que proporciona una URL pública HTTPS para acceder al frontend.

El backend configura CORS para permitir las solicitudes desde los orígenes locales autorizados y desde el origen HTTPS utilizado durante la demostración. Esta configuración permite integrar el recorrido de compra con el retorno de Mercado Pago sin modificar la separación entre frontend y backend.

Conservamos **dos pasos HTTP para la compra**: validación y resumen, seguidos de la confirmación con el medio de pago. Esta separación acompaña la decisión de seleccionar la forma de pago después de revisar el resumen.

La fecha de inicio de la sesión recibida por JSON se convierte a `Date` antes de validarla. Las respuestas satisfactorias utilizan HTTP 200. Los errores se devuelven con su mensaje y el `statusCode` indicado por el error, o 400 como valor predeterminado.

La confirmación en efectivo devuelve `urlRedireccion: "resultado.html"`. Para Mercado Pago devuelve, además de la compra registrada, `idPreferencia` y la URL de Checkout Pro. El endpoint de estado valida que el identificador sea un entero positivo seguro y responde con `{ compra, urlRedireccion: null }`.

La pantalla de resultado consulta al backend utilizando el identificador recibido en el retorno o el de la compra conservada en `sessionStorage`. Los parámetros de retorno sirven para identificar la operación; la aprobación se determina con la verificación del servidor.

El contrato de comunicación puede consultarse en Documentación de endpoints.

Funcionalidad central de compra

La carpeta **`ticket/`** contiene `comprarEntradas()`, `confirmarCompra()` y sus tests. Separamos la preparación del resumen de la operación que registra la compra y asigna sus estados.

`comprarEntradas(usuario, datosCompra)` es una función asíncrona que coordina, en este orden:

1. Validar que el usuario pertenezca al conjunto de usuarios registrados de prueba.
2. Validar la fecha de visita y la apertura del parque.
3. Validar la cantidad de entradas.
4. Validar la presencia, cantidad y contenido de las edades y los tipos de pase.
5. Consultar las entradas acumuladas del mismo usuario para esa fecha y validar el límite.
6. Calcular el monto y devolver los datos de compra junto con `cliente` y `montoTotal`.

Este orden informa el primer error detectado y evita calcular precios o continuar con operaciones dependientes de datos inválidos. Los validadores se reutilizan desde la función principal en lugar de duplicar sus reglas.

Los datos de la visita se representan con `fechaVisita`, `cantidadEntradas`, `edadesVisitantes` y **`tipoPase`**. Los dos últimos son arreglos: cada posición identifica la edad y el pase de un mismo visitante. La cantidad de elementos de ambos debe coincidir exactamente con la cantidad de entradas.

Regla	Decisión adoptada

Regla	Decisión adoptada
Usuario y sesión	Usuarios registrados precargados y sesión simulada activa, de acuerdo con el alcance autorizado.
Fecha	Día actual o futuro, con anticipación máxima de **30 días exactos**, incluyendo el límite. Se rechazan fechas inexistentes y formatos inválidos.
Apertura	Parque cerrado los lunes, el 25 de diciembre y el 1 de enero. Horario informado: de 8:30 a 19:00.
Cantidad por solicitud	Número entero entre 1 y 10.
Límite acumulado	Máximo de 10 entradas por usuario y fecha, sumando las compras registradas y la nueva solicitud. Este control es distinto de un cupo general del parque; **no existe cupo diario general**.
Edades	Valores enteros no negativos, uno por visitante.
Pases	`REGULAR` o `VIP`, seleccionados por visitante; se admiten compras mixtas. La denominación visible acordada es **Entrada Regular**.
Forma de pago	`Efectivo` o `Mercado Pago`, seleccionada después del resumen.

Los catálogos de pases y precios se concentran en `tiposPase.mock.js`. Los importes se expresan en **pesos argentinos (ARS)**:

Tipo de pase	Precio base
Entrada Regular (`REGULAR`)	$10.000
Entrada VIP (`VIP`)	$20.000

Edad del visitante	Descuento sobre el precio base
De 0 a 3 años inclusive	100 %
De 4 a 15 años inclusive	50 %
De 16 a 59 años inclusive	0 %
Desde los 60 años	50 %

`calcularMonto()` recorre los pases, obtiene el descuento de la edad en la misma posición y suma los importes individuales. Una compra cuyos visitantes no pagan conserva un **monto total de cero** como resultado válido. El importe mostrado antes de consultar la API es estimativo; el resumen utiliza el total calculado por el backend.

`confirmarCompra(compra, formaPago)` valida el medio de pago, asigna estados iniciales y guarda la compra mediante el Repository. El registro recibe un identificador incremental. El correo se coordina después de la persistencia y según el medio de pago.

Medio de pago	**`estadoCompra`** inicial	**`estadoPago`** inicial	Interpretación
Efectivo	`pendiente`	`pendiente`	Compra registrada para pagar en boletería; no implica cobro realizado.
Mercado Pago	`pendiente`	`pending`	Compra registrada a la espera de verificar el pago electrónico.

Separamos **estado de compra** y **estado de pago** para representar ambos procesos sin confundir una reserva o una preferencia creada con un pago aprobado. El cobro y la confirmación presencial en

boletería corresponden a otra historia de usuario.

La sesión se representa con usuario, fecha y hora de inicio y fecha y hora de fin. `validarSesion()` exige usuario asociado, inicio válido no futuro y `fechaHoraFin` igual a `null` ; ante una condición inválida informa **«No hay una sesión activa»**. En el frontend se reutiliza la sesión almacenada si existe o se obtiene una sesión de demostración con un usuario precargado. Esta simulación permite probar la compra sin implementar registro y autenticación reales.

Middlewares

Los middlewares necesarios se configuran en **`server.js`** . Utilizamos `express.json()` para interpretar cuerpos JSON y `cors` para permitir el acceso desde los orígenes locales del frontend: `http://localhost:5500` y `http://127.0.0.1:5500` , junto con el origen HTTPS de ngrok configurado para el retorno de Mercado Pago.

El manejo de errores de las rutas se realiza con bloques `try/catch` , que devuelven una respuesta JSON. No se introduce una carpeta independiente de middlewares ni una capa adicional de controladores mientras el alcance pueda resolverse de forma clara con esta organización.

Justificación de la estructura

Esta organización permite mantener **cada funcionalidad junto con sus tests**, identificar sus responsabilidades y facilitar el trabajo paralelo. Centralizar la coordinación en `ticket.service.js` y las reglas de entradas en `ticket.validator.js` evita dispersar o duplicar validaciones.

Separar Repository, correo y Mercado Pago permite sustituir esas dependencias en los tests unitarios. El cliente de Mercado Pago se recibe como dependencia para controlar sus respuestas, y su inicialización con credenciales se concentra en un módulo específico.

La estructura prioriza la simplicidad del TP6. El acceso a datos queda encapsulado para permitir reemplazar el almacenamiento sin trasladar operaciones de archivos a la lógica de compra. Las rutas concentradas en `server.js` y las páginas sin framework requieren mantener disciplina para evitar duplicación cuando se agregan comportamientos.

6. Herramientas del frontend

La interfaz de EcoHarmony Park se implementa como una **aplicación web responsive con HTML, CSS y JavaScript**, organizada por pantallas. La demostración puede ejecutarse localmente.

Tecnologías utilizadas

Herramienta	Utilización
HTML5	Estructura de páginas, formularios, resumen y resultado.
CSS3	Identidad visual, distribución y adaptación a distintos tamaños de pantalla.
JavaScript	Interacción, validación de interfaz, actualización del DOM y navegación.
Fetch API	Comunicación HTTP y JSON con el backend.

Herramienta	Utilización
sessionStorage	Conservación de la compra durante el recorrido entre pantallas de la pestaña.
localStorage	Consulta de una sesión simulada almacenada cuando existe.
Google Fonts — Lato	Tipografía definida para el proyecto.

Las páginas se sirven durante el desarrollo mediante un servidor estático local en el puerto 5500, compatible con los orígenes habilitados en el backend.

Durante el desarrollo, las páginas del frontend se sirven mediante un servidor estático local en el puerto 5500. Para verificar el recorrido de pago con Mercado Pago Checkout Pro, utilizamos ngrok para proporcionar una dirección pública HTTPS que permita regresar a la aplicación después de interactuar con el proveedor de pagos.

Justificación de las herramientas

HTML, CSS y JavaScript. Permiten implementar el recorrido acotado de páginas del TP6 y trabajar directamente con formularios y eventos del navegador. Se adoptó esta solución por su sencillez de puesta en marcha y por permitir distribuir el trabajo por pantallas sin incorporar una configuración adicional de compilación.

React, Vite, Bootstrap y Tailwind fueron alternativas iniciales. La implementación utiliza las capacidades del navegador y estilos propios, por lo que esas herramientas no forman parte de la solución adoptada. El intercambio entre pantallas requiere coordinar explícitamente los datos y mantener los elementos comunes para evitar duplicación.

Fetch API. Permite solicitar los catálogos y enviar los datos de compra. Las validaciones visuales mejoran la experiencia de uso, pero **no reemplazan las validaciones de negocio del servidor**. El resumen utiliza la respuesta del backend para mostrar el monto calculado.

Almacenamiento del navegador. `sessionStorage` conserva `compraEnCurso` y `compraConfirmada` durante la navegación de la misma pestaña. No sustituye la persistencia del backend. `localStorage` permite reutilizar una sesión simulada si fue almacenada; el acceso de demostración también puede proporcionar una sesión con un usuario precargado.

Identidad visual. Utilizamos la tipografía **Lato**, el logotipo aprobado por los PO y la paleta de colores del proyecto: `#386641`, `#6A994E`, `#A7C957`, `#F2E8CF` y `#BC4749`. La interfaz prioriza tonos verdes y crema y mantiene coherencia entre formularios, mensajes y navegación.

Diseño responsive

La decisión de producto es ofrecer una **web responsive con criterio mobile first**, utilizable desde teléfonos, tablets y computadoras. El diseño aprobado contempla versiones de escritorio y móvil.

La implementación adapta navegación, distribución de formularios, calendario, tarjetas de visitantes y resumen mediante CSS y media queries. Se presta atención a la legibilidad, el tamaño de los controles y la claridad de las acciones y mensajes.

La validación manual considera diferentes anchos de pantalla y el recorrido completo de compra. La disposición concreta de campos, calendario y mensajes se resuelve por el equipo priorizando la experiencia de uso, conforme a lo acordado con los PO.

Criterios de implementación

El frontend sigue la división de tareas acordada y **no utiliza TDD**. Se valida principalmente de manera manual y se conecta progresivamente con la API.

El formulario solicita fecha, cantidad, edad y pase por visitante. Presenta un total estimado mientras se completan los datos y muestra el resumen calculado por el servidor antes de continuar al pago. La selección del medio de pago se realiza en la pantalla siguiente.

Al confirmar, se deshabilitan temporalmente los botones para evitar solicitudes repetidas desde esa pantalla. `pago.js` conserva la respuesta y navega a la URL indicada por el backend: el resultado local para efectivo o Checkout Pro para Mercado Pago.

La pantalla final consulta el estado al backend y muestra identificador, fecha, cantidad, importe y estados de la operación. Para efectivo informa el pago en boletería; para Mercado Pago utiliza la compra devuelta por la verificación del servidor. Si la consulta falla, presenta el error y no anuncia una aprobación.

Los PO aprobaron el logotipo y el prototipo y dejaron a criterio del equipo la disposición de campos, el calendario y los mensajes, priorizando la experiencia de uso. La denominación acordada es **Entrada Regular** y la selección del medio de pago se mantiene después del resumen.

Las imágenes y pantallas informativas acompañan la identidad del parque. No amplían el alcance funcional hacia la inscripción a actividades, el registro de usuarios o la administración del parque.

7. Herramientas del backend

Utilizamos **Node.js + Express**, con JavaScript, para implementar las funcionalidades del backend y la comunicación con el frontend y los servicios externos. El código utiliza módulos **CommonJS** con `require` y `module.exports`.

Tecnologías utilizadas

Herramienta	Utilización
Node.js	Ejecución de JavaScript y acceso al sistema de archivos.
Express 5	Aplicación HTTP, middlewares y endpoints.
JavaScript	Validaciones, cálculo del monto y coordinación de la compra.
cors	Habilitación de los orígenes locales y del origen configurado para ngrok.
Módulos nativos `node:fs` **y** `node:path`	Persistencia JSON y construcción de rutas de archivos.
SDK `mercadopago`	Creación de clientes oficiales de preferencias y pagos.
npm	Administración de dependencias, versiones y scripts.

Justificación de las herramientas

Node.js. Permite utilizar JavaScript también en el servidor y trabajar con las herramientas de testing elegidas. Los servicios de compra y las integraciones exponen funciones asíncronas para coordinar operaciones y propagar sus errores con `async/await`.

Express. Facilita recibir solicitudes HTTP y devolver JSON sin imponer una arquitectura compleja. `server.js` declara las rutas y delega la lógica en los servicios; `index.js` inicia el servidor. La separación mantiene independiente la configuración de la aplicación y la apertura del puerto HTTP.

JavaScript. Se utiliza para las reglas de negocio, el cálculo y la coordinación. Los validadores tienen entradas y resultados concretos y lanzan errores con mensajes específicos. Las funciones auxiliares permiten reutilizar conversiones de fecha, comprobaciones de cantidades y descuentos por edad.

Repository y JSON. Las compras se conservan en `src/data/compras.json` para que sobrevivan al reinicio de la aplicación. Esta alternativa satisface la persistencia requerida en una demostración local sin instalar un servidor de base de datos.

npm. Las dependencias y sus versiones se registran en `package.json` y `package-lock.json`. Los scripts permiten iniciar el backend con `npm start`, ejecutar los tests y utilizar `npm run test:watch` desde `backend/`. La configuración de ejecución utiliza variables de entorno cargadas desde `.env.gmail` mediante la opción `--env-file` de Node.js. Desde la raíz del proyecto también se pueden ejecutar los scripts mediante `npm --prefix backend`.

Organización y criterios de implementación

El backend sigue la estructura por responsabilidades y se desarrolla mediante **TDD**, respetando RED → GREEN → REFACTORING y el caso de Verificación y regresión.

Las validaciones se ejecutan en un orden explícito. Las reglas de fecha trabajan por día de calendario, sin comparar la hora de la visita; el límite de anticipación se calcula sumando **30 días**, no un mes calendario. Los datos recibidos por HTTP se convierten cuando corresponde antes de llamar a las funciones de dominio.

`CompraRepository` recibe la ruta del archivo en su constructor y ofrece `guardarCompra()`, `obtenerCompraPorId()`, `actualizarEstadosCompra()` y `obtenerEntradasAcumuladas()`. Si el archivo todavía no existe, la lectura devuelve una colección vacía. Al guardar, se asigna un identificador igual al máximo existente más uno y se conservan las compras anteriores.

La consulta de acumulados filtra por identificador de usuario y fecha y suma las cantidades registradas, incluidas las compras con estados pendientes. No aplica un cupo global del parque. Las actualizaciones de estado conservan el identificador y los demás datos de la compra.

La persistencia utiliza lecturas y escrituras síncronas de un archivo JSON, aunque los métodos del Repository se consumen con una interfaz asíncrona. Es una solución acotada a la demostración local: no proporciona transacciones, bloqueo entre procesos ni garantías de concurrencia propias de una

base de datos. Encapsularla permite cambiar el almacenamiento sin dispersar operaciones de archivos por los servicios.

Se procura que los efectos externos ocurran después de validar los datos. Los errores de persistencia se propagan; los fallos de correo se tratan después del registro para conservar la compra.

8. Herramientas de testing

Para implementar y ejecutar las pruebas unitarias del backend utilizamos **Jest**, con mocks para controlar las dependencias de cada función o servicio.

Herramienta seleccionada

Herramienta	Utilización
Jest 30	Pruebas unitarias, aserciones, mocks y control del reloj.
Archivos de prueba exclusivos	Verificación de persistencia sin modificar las compras utilizadas en la demostración.

Justificación de Jest

Elegimos Jest porque reúne las funciones necesarias para escribir, organizar y ejecutar tests del backend en JavaScript. Su uso es adecuado para TDD, ya que permite comprobar el resultado de cada etapa **RED** → **GREEN** → **REFACTORING**.

Verificamos funciones como `validarUsuario()`, `validarSesion()`, `validarFecha()`, `validarCantidadEntradas()`, `validarCamposObligatorios()`, `calcularMonto()` y las operaciones de los servicios y del Repository.

Los **mocks** permiten simular respuestas de correo, pagos y almacenamiento, comprobar llamadas y orden de operaciones y evitar depender de servicios externos en cada ejecución. El reloj controlado permite probar fechas actuales, pasadas, futuras y límites de anticipación con datos reproducibles.

Aplicación durante el desarrollo

Las pruebas cubren casos válidos, datos faltantes, formatos inválidos, valores límite y errores de las dependencias. Entre los límites se incluyen 0, 1, 10 y 11 entradas; 30 y 31 días de anticipación; y edades 3/4, 15/16 y 59/60.

Antes de registrar una fase se ejecuta el test correspondiente junto con los anteriores relacionados. La implementación acumulada debe conservar los comportamientos existentes; un resultado inicialmente satisfactorio se registra como Verificación y regresión cuando no exige cambios de código.

Las pruebas del Repository utilizan rutas de archivos propias y limpieza de sus datos. Los tests unitarios de correo usan un transporte simulado y los de Mercado Pago reciben clientes mock para controlar sus respuestas y los errores de comunicación.

La instalación de dependencias se reproduce con `npm ci` en `backend/`. Para ejecutar las pruebas unitarias de un servicio se indica a Jest el archivo correspondiente. Las comprobaciones de la interfaz se realizan manualmente y no se presentan como TDD del frontend.

Las verificaciones manuales de integración permiten comprobar el funcionamiento conjunto del frontend, el backend y los servicios externos. Estas actividades complementan las pruebas unitarias, pero no se presentan como tests automatizados ni se incorporan artificialmente al ciclo TDD.

9. Herramientas para el manejo de correos electrónicos

Para el correo de confirmación utilizamos **Nodemailer y Handlebars**, separando la generación del contenido, la validación del destinatario y el envío. **Mailtrap** permite capturar mensajes de desarrollo y **Gmail SMTP** ofrece otra configuración de transporte para la entrega.

Tecnologías utilizadas

Herramienta	Utilización
Nodemailer	Envío de correos desde Node.js mediante SMTP.
Handlebars	Generación de contenido HTML a partir de una plantilla.
Mailtrap	Captura y visualización de correos de desarrollo.
Gmail SMTP	Configuración de transporte para entregar el resumen de compra.
Jest y mocks	Pruebas unitarias de contenido, destinatario, envío y tratamiento de errores.

Justificación de las herramientas

Nodemailer. Permite enviar desde el backend utilizando un transporte SMTP configurable. El servicio de correo recibe los datos de una compra registrada y mantiene el envío separado de la validación y del cálculo del monto.

Handlebars. La plantilla `confirmacionCompra.hbs` separa presentación y lógica. Se carga y compila una sola vez y contempla saludo, identificador, fecha, cantidad de entradas, tipos de pase, medio de pago e importe. Las funciones auxiliares formatean los valores recibidos para su presentación. El logo se incorpora como adjunto referenciado mediante CID.

Mailtrap. Permite capturar y revisar mensajes de desarrollo sin entregarlos a los destinatarios habituales. Facilita inspeccionar contenido y formato.

Gmail SMTP. Permite utilizar el mismo servicio de envío con otra configuración de transporte. Los parámetros del servidor y las credenciales se proporcionan mediante variables de entorno.

Configuración SMTP. El servicio de correo utiliza una configuración de transporte mediante variables de entorno, lo que permite establecer los parámetros de conexión y las credenciales sin incorporarlos directamente al código fuente.

Durante la integración se utilizó Gmail SMTP para verificar la entrega efectiva de los correos de confirmación. Las variables `SMTP_HOST`, `SMTP_PORT`, `SMTP_USER`, `SMTP_PASS` y `SMTP_FROM` se cargan

desde el archivo `.env.gmal`, manteniendo las credenciales fuera del control de versiones.

Esta configuración permite modificar el servicio de correo utilizado sin alterar la lógica de generación del mensaje ni el proceso de confirmación de la compra.

El destinatario es **el correo de la cuenta registrada**; no se solicita una dirección alternativa durante la compra. La validación del servicio comprueba que el valor exista y contenga `@`. Es una validación básica adoptada para el alcance del TP6, no una comprobación completa de la existencia del buzón.

Integración con el proceso de compra

1. El backend valida y registra la operación correspondiente mediante el Repository.
2. Con **Efectivo**, utiliza los datos guardados para enviar la confirmación del registro. El pago continúa pendiente en boletería.
3. Con **Mercado Pago**, el correo de confirmación de pago corresponde únicamente después de verificar la aprobación y persistir los estados resultantes.
4. Handlebars genera el resumen HTML y Nodemailer lo envía al correo del usuario registrado.
5. Si el envío falla, el error se registra sin borrar la compra ni revertir un pago confirmado.

La persistencia y el envío son operaciones distintas: **no se envía una confirmación de una compra que no pudo guardarse** y un error de correo no invalida el registro. La consulta repetida de un pago ya confirmado debe conservar su estado y evitar correos duplicados.

El mensaje resume la información de la compra. Su recepción no convierte un registro en efectivo en un pago realizado; la información mostrada debe respetar el estado de la operación.

Testing y criterios de implementación

Con **Jest** verificamos la generación de HTML, los datos previstos por la plantilla, el formato de fecha e importe, el destinatario y la coordinación del envío. Un transporte mock permite comprobar las opciones de Nodemailer y simular errores sin conectarse a SMTP.

La previsualización HTML permite revisar el contenido y la presentación del mensaje. La configuración de envío utiliza variables como `SMTP_HOST`, `SMTP_PORT`, `SMTP_USER`, `SMTP_PASS` y `SMTP_FROM`. Las credenciales deben mantenerse privadas y fuera del control de versiones.

Los PO permiten simular el correo. Si se utiliza esa modalidad en la demostración, el mensaje debe presentarse en una ventana aparte. Las pruebas unitarias con mocks y una previsualización local no se confunden con la entrega efectiva de un correo.

10. Integración con Mercado Pago

El pago electrónico utiliza **Mercado Pago Checkout Pro**, con preparación de preferencias y consulta de pagos desde el backend mediante el **SDK oficial de Node.js**. La lógica se encapsula en un servicio que recibe los clientes como dependencias.

La conexión con el recorrido de compra comprende la creación de la preferencia, la redirección a Checkout Pro, el retorno al sitio y la consulta del estado desde la pantalla de resultado. La demostración se configura con credenciales y operaciones de prueba, sin transacciones monetarias reales.

Tecnologías previstas

Herramienta	Utilización
Checkout Pro	Flujo externo de pago y retorno al sitio.
SDK oficial `mercadopago` para Node.js	Inicialización de `MercadoPagoConfig`, `Preference` y `Payment`.
API de preferencias de pago	Creación de la operación asociada al importe de la compra.
API de consulta de pagos	Búsqueda y verificación del pago por referencia de compra.
Clientes recibidos como dependencias	Operaciones `create` y `search`, sustituibles por mocks en los tests unitarios.
ngrok y `PUBLIC_URL`	Exposición temporal del frontend mediante HTTPS y construcción del retorno desde Checkout Pro.
Credenciales, cuentas y medios de pago de prueba	Demostración del recorrido sin cobros reales.

La decisión de integración utiliza **consulta de estado desde el backend, sin webhooks**. Esto permite una demostración local sin mantener un receptor público permanente de notificaciones.

Justificación de las herramientas

Preferencias de pago. Cada preferencia representa el total de una compra persistida mediante un único ítem: título «Entradas EcoHarmony Park», `quantity: 1`, `unit_price` igual a `montoTotal` y `currency_id: "ARS"`. La cantidad del ítem de pago no representa la cantidad de visitantes; el detalle de entradas permanece en la compra.

La propiedad **`external_reference`** contiene el identificador de compra convertido a texto. De esta manera, las consultas del proveedor pueden relacionarse con el registro local sin depender del contenido de la URL de retorno.

SDK oficial y clientes recibidos como dependencias. `crearClientesMercadoPago()` valida la presencia de `MP_ACCESS_TOKEN` e inicializa los clientes de preferencias y pagos. `crearPreferenciaPago()` delega en `clienteMP.create({ body: preferencia })`; la búsqueda utiliza `clienteMP.search()` con la referencia externa y orden por fecha de creación descendente. Separar la configuración de las funciones de negocio permite probarlas con clientes simulados.

Checkout Pro. Proporciona una interfaz externa para el pago sin construir un formulario propio de tarjetas. De la respuesta de creación se obtienen el identificador de preferencia y `init_point`, utilizado como URL de redirección.

Retorno HTTPS. `PUBLIC_URL` identifica la dirección pública del frontend expuesto mediante ngrok. El backend comprueba que exista y sea una URL HTTPS válida y construye `resultado.html?id=[identificador]`. Las tres propiedades de `back_urls` — `success`, `failure` y `pending` — apuntan a ese mismo resultado, porque la aprobación se determina consultando al servidor.

Consulta y verificación del pago. `consultarEstadoCompraMP()` recupera la compra persistida y, cuando corresponde a Mercado Pago, delega en `confirmarPagoMercadoPago()` la búsqueda y validación de los pagos asociados mediante `external_reference`.

Para confirmar la operación, el backend verifica que exista un pago con estado `approved` y comprueba la coincidencia del identificador de compra, el importe y la moneda (`ARS`).

Si las verificaciones son satisfactorias, actualiza `estadoCompra` a `confirmada` y `estadoPago` a `approved`, persiste los cambios y posteriormente intenta enviar el correo de confirmación.

La pantalla de resultado consulta este estado al cargarse. La aprobación se determina exclusivamente mediante la verificación realizada por el backend, sin confiar en los parámetros de retorno de Checkout Pro ni depender de notificaciones por webhook.

Conservación del resultado. Una compra que ya tiene `estadoCompra: "confirmada"` y `estadoPago: "approved"` se devuelve sin repetir la búsqueda, la actualización ni el correo. Este control evita duplicar esos efectos en consultas posteriores del mismo registro.

Integración con el proceso de compra

1. El visitante completa los datos de las entradas y revisa el resumen con el monto calculado por el backend.
2. Selecciona **Mercado Pago** y envía la confirmación a `POST /api/confirmacion`.
3. La compra se guarda con estado de compra `pendiente` y estado de pago `pending`.
4. El backend inicializa el cliente de preferencias y crea una preferencia asociada al identificador y al monto de la compra guardada, con sus URL de retorno.
5. La respuesta incluye `idPreferencia` y `urlRedireccion`; el navegador continúa hacia Checkout Pro.
6. Al volver a `resultado.html`, el frontend obtiene el identificador de la operación y consulta `GET /api/pagos/:id/estado`.
7. El backend recupera la compra y busca sus pagos por referencia externa. Selecciona un pago con estado `approved` y verifica coincidencia de **referencia, importe y moneda ARS**.
8. Si las comprobaciones son válidas, actualiza la compra a `confirmada` y el pago a `approved`. Primero persiste y después intenta enviar el correo.
9. La pantalla muestra la compra devuelta por el backend. Si ya estaba confirmada, las consultas posteriores conservan el resultado sin repetir el correo.

Crear una preferencia o regresar desde Checkout Pro no equivale a un pago aprobado. Si no existe un pago aprobado válido, la consulta conserva los estados registrados. Los resultados pendientes, rechazados o con datos inconsistentes no habilitan una confirmación de pago.

Los registros se conservan en el historial y participan del cálculo de entradas acumuladas. El visitante puede comenzar una nueva operación; el recorrido de consulta no elimina automáticamente las compras anteriores.

Los errores de configuración, creación de preferencias o consulta se propagan a las rutas para informar el problema. Una falla posterior a guardar la compra no elimina el registro. Un error del correo se registra sin revertir los estados ya persistidos.

Restricciones de seguridad

- No utilizar credenciales de producción.
- No utilizar tarjetas ni cuentas reales para ejecutar pagos.
- No realizar transacciones monetarias reales.
- No recopilar ni almacenar información sensible de tarjetas.
- Mantener las credenciales privadas fuera del repositorio, mediante variables de entorno.
- Utilizar mecanismos oficiales de prueba o la simulación autorizada por los PO.
- Verificar desde el backend el estado, importe, moneda y asociación del pago antes de confirmar la compra.
- No aceptar parámetros de retorno del navegador como prueba suficiente de aprobación.
- Evitar que las consultas posteriores de una compra confirmada generen correos duplicados.

`MP_ACCESS_TOKEN` se utiliza exclusivamente en el backend. `PUBLIC_URL` configura el retorno HTTPS; el origen público utilizado se contempla en CORS para que la página de resultado pueda consultar la API local durante la demostración.

11. Uso de Inteligencia Artificial

Durante el desarrollo del TP6 utilizamos herramientas de **Inteligencia Artificial (IA)** como apoyo para la organización, implementación, revisión y documentación de las funcionalidades correspondientes a la historia de usuario **«Comprar entradas»** de EcoHarmony Park.

Su utilización permitió realizar consultas técnicas, analizar alternativas de implementación y resolver problemas encontrados durante el desarrollo, sin reemplazar las decisiones del equipo ni la verificación efectiva de las funcionalidades.

Aplicaciones previstas

Aplicaciones de la Inteligencia Artificial

- **Investigación y consultas técnicas:** comprensión de herramientas, documentación y alternativas de implementación.
- **Asistencia en la implementación:** generación y revisión de sugerencias de código para frontend y backend.
- **Apoyo durante TDD:** interpretación de errores, análisis de resultados y consultas sobre las fases RED, GREEN y REFACTORING.
- **Detección y resolución de errores:** identificación de posibles causas y evaluación de alternativas de solución.
- **Revisión de código:** sugerencias para mejorar la legibilidad, organización y mantenimiento.

- **Documentación:** asistencia en la organización, redacción y revisión de los documentos del TP6.

Criterios de utilización

- Las sugerencias de IA deberán ser **revisadas y comprendidas** antes de incorporarse.
- El código sugerido se adaptará a los requerimientos y se verificará mediante tests o validaciones manuales.
- No se utilizará IA para inventar reglas de negocio pendientes de confirmación por los Product Owners.
- Las decisiones de diseño y la aprobación de resultados serán responsabilidad del grupo.
- No se compartirán credenciales, claves ni información sensible con herramientas de IA.

Relación con la metodología

La Inteligencia Artificial se utilizó siguiendo el enfoque **Just-in-Time**, recurriendo a ella ante necesidades concretas del desarrollo.

En el backend, sirvió como apoyo para analizar errores, comprender resultados de pruebas y revisar implementaciones, sin sustituir la ejecución real de los tests ni la evidencia de las fases TDD.

En el frontend, permitió consultar alternativas de implementación y resolver dificultades relacionadas con la interfaz y la comunicación con el backend, verificando posteriormente los resultados mediante pruebas manuales.

Las sugerencias obtenidas fueron consideradas herramientas de asistencia. La responsabilidad sobre las decisiones técnicas, las reglas de negocio y la validación de los resultados permaneció en el equipo de desarrollo.